TRENTOS-M RPi_SD

Resources

Driver Code

- Driver for integrated SD controller (also EMMC): https://github.com/rsta2/circle
- Also use mailbox, environment and general component structure according to: ssh://git@bitbucket.hensoldt-cyber.systems:7999/sc/nic_rpi.git

Datasheets

- SDHost Controller spec: http://academy.cba.mit.edu/classes/networking_communications/SD/SD.pdf
- BCM2835 TRM: https://www.raspberrypi.org/app/uploads/2012/02/BCM2835-ARM-Peripherals.pdf

Other resources

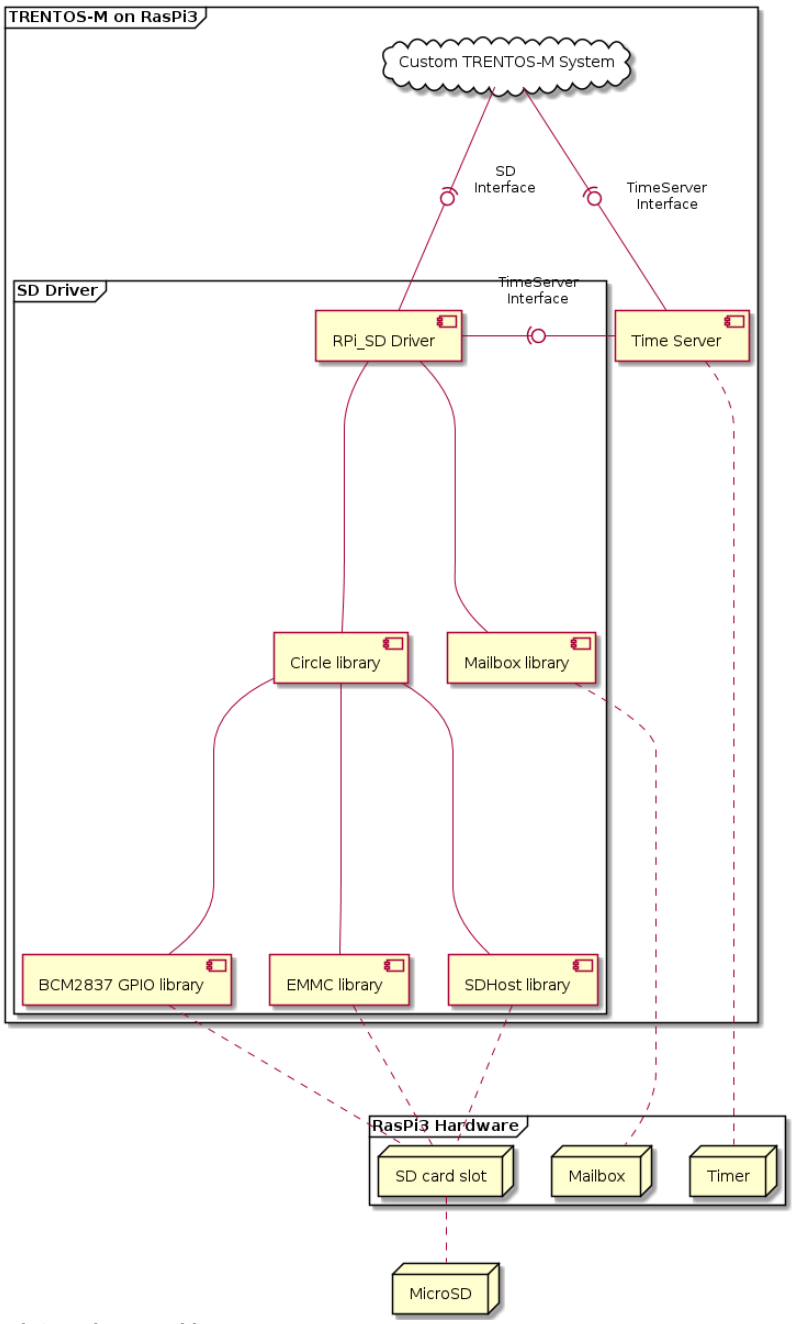
- More information on SD card: https://yannik520.github.io/sdio.html
- Mailbox: https://github.com/raspberrypi/firmware/wiki/Mailbox-property-interface and https://github.com/raspberrypi/documentation/blob/JamesH65-mailbox_docs/configuration/mailboxes/propertiesARM-VC.md

Overview

The RPi_SD component is basically a combination of a redesigned version of the RPi_NIC component, the circle library, the upper-level SD card driver of the RPi_SPI_SD component and the bcm2837 GPIO files used in the RPi_SPI_Flash component.

The RPi_NIC component was used to get the mailbox functionality and general environment functions, as well aim at a similar overall component structure. Instead of using the *uspi* library, the *circle* library was used instead. The GPIO files were used to have a way to configure the GPIO pins for the SDHost controller or the EMMC controller. On top of the gpio files, files from the *circle* library were used to enable general emmc/sdhost functionality. The SD card driver from the RPi3 flash component was adapted and built on top.

The component offers two operating modes: EMMC mode and SDHost mode. These modes can be switched with a macro variable call *USE_SDHOST*. If defined, the driver runs in SDHost mode, if not defined, in EMMC mode. The default is SDHost mode, thus *USE_SDHOST* is defined.



RPi_SD Driver - Architecture

For more information on implementation details and findings along the way, refer to ticket SEOS-855 - Getting issue details... STATUS in Jira.

SD interface implementation

The storage interface implementation is based on the one from RPi_SPI_SD component, modified to use the functions from the new driver.

Furthermore, interface handlers are implemented for the SDHost mode as well as EMMC mode. Initializing the SD card must be take place after the *post_init()* function, as otherwise interrupts would not be correctly connected.

Limitation

The component is only tested for RPi3 B+, but should also work for other Raspberry Pis. Furthermore, only SDHost mode is working, the EMMC mode is not tested thoroughly and should not be used with normal SD cards.

The driver currently has terrible performance and should be looked into for the future.

Hardware

In order to find information on how to communicate with the SD card slot, we need to refer to dts files.

Resources

- TRENTOS-M: seos_test/seos_sandbox/sdk-sel4-camkes/kernel/tools/dts/rpi3.dts
- Linux Kernel: arch/arm/boot/dts/bcm283x.dtsi
- U-Boot: arch/arm/dts/bcm283x.dtsi

Findings

It turns out that the internal SD card slot is mapped to GPIO pins 48-53. Depending on the alternative function that is set for these pins, the SD card slot behaves as EMMC reader (Alternative Function 3) or SD card reader (Alternative Function 0). This can be explicitly set in the *config.txt* boot file for the raspi (https://www.raspberrypi.org/documentation/configuration/config-txt/gpio.md) or done through the GPIO library. Pull modes for the pins can only be configured via the *config.txt* file.

Usage

CMake

RPi_SD Driver - Usage 1

```
DeclareCAmkESComponent_RPI_SD_DRV (    <TypeNameSD>    LIBS        <TimeServerClient>)
```

CAmkES

RPi_SD Driver - Usage 2

```
import <if_OS_Timer.camkes>;#include "RPI_SD/RPI_SD.camkes";DECLARE_COMPONENT_SYSTIMER_HW(SYSTIMER)
DECLARE_COMPONENT_MAILBOX_HW(MAILBOX)DECLARE_COMPONENT_SDHOST_HW(SDHOST)DECLARE_COMPONENT_EMMC_HW(EMMC)
DECLARE_COMPONENT_RPI_SD_DRV(SD)#include "TimeServer/camkes/TimeServer.camkes"DECLARE_COMPONENT_TimeServer
(TimeServer)assembly {    composition {        // SD        DECLARE_AND_CONNECT_INSTANCE_SD_DRV_HW
(            MAILBOX, mailbox,             SDHOST, sdhost,             EMMC, emmc,            SYSTIMER,
systimer,            SD, sd        )                // demo application                component
<TypeMyApplication>  <InstanceMyApplication>;        connection  seL4RPCCall         tester_msd_storage
(from <InstanceMyApplication.rpc>,  to sd.storage_rpc);        connection  seL4SharedData
tester_msd_storage_port(from <InstanceMyApplication.dataport>, to sd.storage_port);
//------------------------------------------------------------------------        // TimeServer
//------------------------------------------------------------------------
DECLARE_AND_CONNECT_INSTANCE_TimeServer(            TimeServer,            timeServer,            sd.
timeServer_rpc, sd.timeServer_notify        )    }configuration {        CONFIGURE_INSTANCE_SYSTIMER_HW
(systimer, 0x3F003000, 0x1000)              CONFIGURE_INSTANCE_MAILBOX_HW(mailbox, 0x3F00B000,
0x1000)               // CONFIGURE_INSTANCE_SDHOST_HW(sdhost, 0x3F202000, 0x100, 88)
CONFIGURE_INSTANCE_SDHOST_HW(sdhost, 0x3F202000, 0x1000, 66)               CONFIGURE_INSTANCE_EMMC_HW(emmc,
0x3F300000, 0x1000, 94)               CONFIGURE_INSTANCE_SD_DRV(sd, 4*40960)    }
```

Platforms

- RPi3
- RPi4 (branch: rpi4_porting)